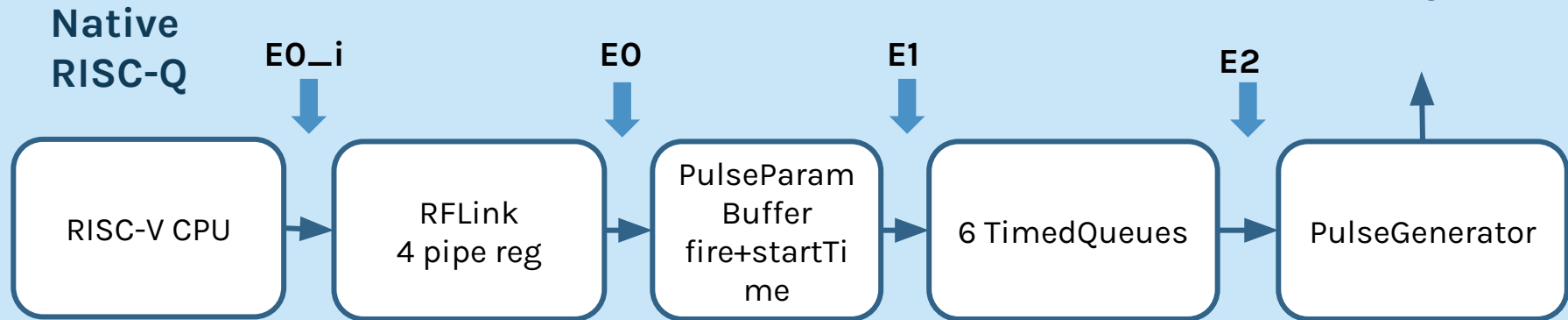
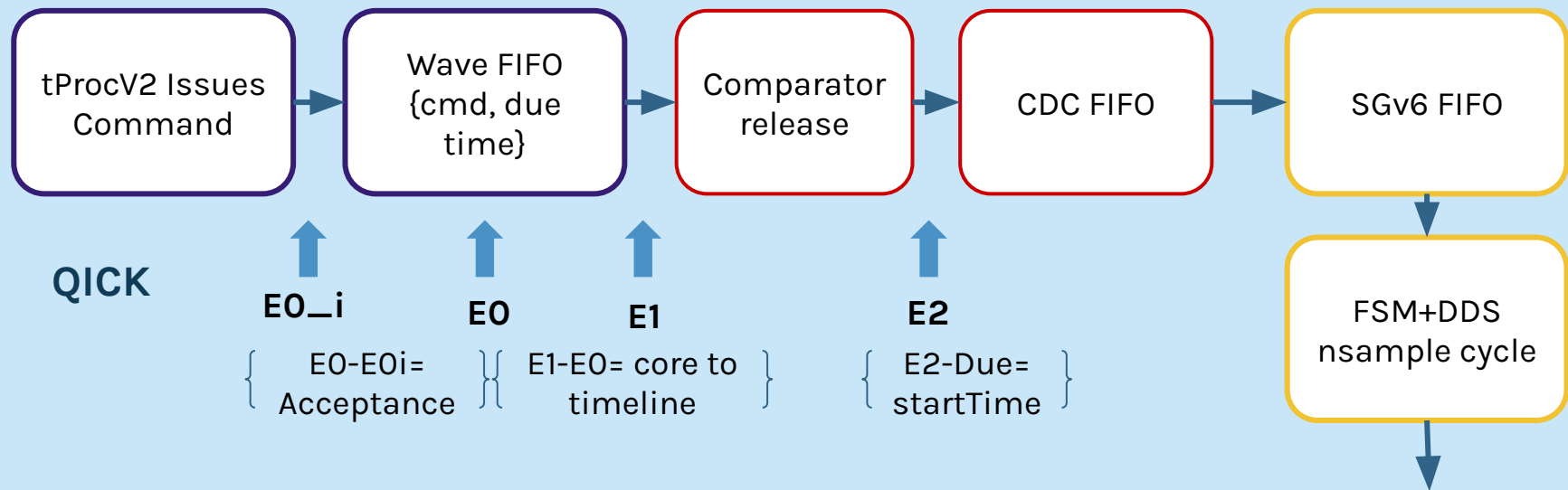


■ **Timing analysis Purpose – V&V team**

Establish Equivalence: Observable timing between tProcV2 with RISC-Q through adapter.

- Acceptance time
- Pulse starts time
- Pulse Sequence order

Pipeline delays + Clock phase jitter/queueing → Measurement required



■ Timing analysis Methods – V&V team

QICK program Python API → QICKEmu + monitor → 1 log line/command → Analysis

QICK test → RISC-Q translate → RISC-Q scala sim + monitor → same log → Compare Ref

	tProc	RISC-Q	Logged
E0_i	Program writes	CPU writes	time, core cycle, , requested time, command
E0	Accepted, due stamped	Fire decoded	time, QICK: due
E1	Reaches timeline	Queued, due stamped	time, clock count, RISC-Q: due
E2	released	released	time, clock count
Tag	Amp	Amp	Per pulse

■ Timing analysis Results- V&V team

Start time: Pulses, 3 gaps, 4 Clock Phases

- Differences: tProc due + 4ticks deterministic, RISC-Q releases early (52-2 cycles before due) to line with own generator

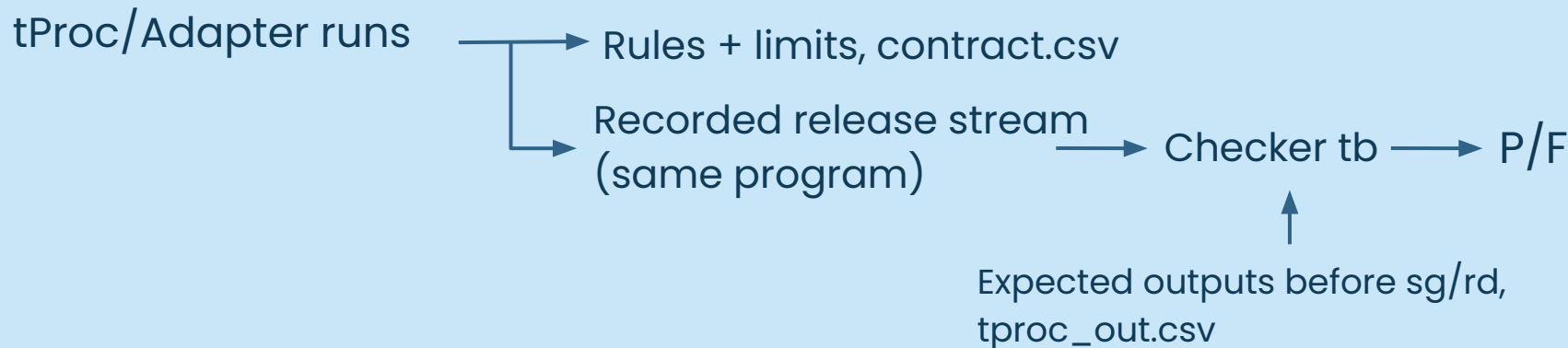
Acceptance Time: 300 pulses queued ahead, pulse written after due

- Differences: tProc holds 256, then pauses (7.1us) none lost, RISC0Q holds ~5, 8queued → 3 silently dropped, CPU never paused
- tPproc sent late pulse in 20ns no flag, RISC-Q sent immediately no flag
- Sgv6: once command released, SGv6 starts 37.7-39.3ns after due, <1 jitter cycle, output 9 cycles later

Sequence Order: later-due pulse written first

- Match: write order, tProc pulse waited 223 ticks

■ Limits, start-up, and testbenches- V&V team



Stress tests

- Initiated: capacity (pause vs drop), late command, write order vs due order, output together (skew)
- Tbc: restart/power-up, slack erosion, feedback latency, long runs